

HIL Benchmark: ORB-SLAM2 vs. OV2SLAM (Accurate)

ROS2 HIL Visual-Servoing Project

2026-07-05

1 Summary

This report compares ORB-SLAM2 and OV2SLAM (accurate profile) on the same hardware-in-the-loop (HIL) flight scenario: an oracle detector + proportional controller flying the same approach trajectory, with the `INITIALIZER.GATE` warm-up enabled for both backends. Three repeated runs per backend (run tag `preflight_code_julien_{1,2,3}`, 2026-07-05) were evaluated by literally invoking the `evo_ape` command-line tool (evo, the same tool the offline EuRoC benchmark study used) as a subprocess – `evo_ape tum gt.tum slam.tum --align --correct_scale` – and parsing its stdout, not a hand-written re-implementation of evo’s alignment/association code. All accuracy figures are reported in meters.

Both backends tracked the full mission successfully in all 6 runs – this is itself a result: OV2SLAM failed to initialize in every attempt prior to this run (see `docs/fixlog/014`, `015` for the diagnosis and fixes that made this data possible).

Important – neither backend is actually flying the drone in these runs. Both configs use `detector.type: oracle` with `controller.use_slam_pose/use_slam_depth` left at their default (`false` – confirmed in `parse_stack.py`). The proportional controller’s control input is the oracle detector’s ground-truth-derived bounding box (`/yolo/detections`), not SLAM. **SLAM is a passive passenger here:** recorded and evaluated for accuracy, but it never influences `/cmd_vel` or the flight path. This is the right setup for an apples-to-apples accuracy/performance comparison (both backends fly the identical, repeatable, SLAM-independent path), but it does *not* demonstrate SLAM-driven visual servoing.

How easy would SLAM-driven control actually be? For the *IBVS* controller (TS1), trivially easy – `controller.use_slam_depth/ use_slam_pose` are real, already-implemented parameters (`visp_servo/src/ibvs_controller.cpp:19-38`), wired all the way from the stack config through the launch file to the controller (see `ov2slam_ibvs.yaml`); switching to SLAM-driven IBVS is a config change, not new code (though it inherits known open caveats – monocular scale drift, background-vs-target depth bias – tracked in `docs/fixlog/009`). *Not* implemented for the **proportional controller (TS2)** used in this report, and there is a concrete reason: TS2’s entire control law (`proportional_controller.cpp:29-41`) is 2D image-plane only – forward velocity from apparent bounding-box size, lateral/vertical velocity from normalised pixel offset. It never needs depth or 3D pose at all, unlike IBVS’s image Jacobian, which requires an explicit depth estimate. Wiring SLAM into TS2 wouldn’t extend it so much as replace its control law with something closer to PBVS (TS4) – a materially different controller, not a flag flip. Separately: “oracle” and “TS2” are independent axes (detector choice vs. controller choice) that already compose freely –

oracle+proportional is exactly what this report benchmarks; there is no such thing as an "oracle controller."

2 ATE RMSE and Tracking Coverage

Two ATE numbers are reported per run. **Mission ATE** restricts evaluation to $t \geq \text{FSM-start}$ (the `/bench/state` marker), excluding the `INITIALIZER_GATE`'s own pre-mission warm-up wiggle – this is the number that reflects actual tracking quality during the flight, and the one used for comparison. **Full-bag ATE** includes that warm-up window too, for transparency; it reads noticeably higher because a handful of high-error points right after SLAM initializes (before scale/pose has settled) get squared into the RMSE despite being a small fraction of the total samples – see the note at the end of this section for the concrete numbers behind that gap.

Sample size caveat: `evo_ape`'s own default association tolerance (`--t_max_diff 0.01s`) is tight relative to GT's ~ 50 Hz and SLAM's ~ 16 – 20 Hz rates, so only pose pairs within 10 ms of each other count – between 32 and 171 pairs per run here (out of hundreds of raw samples each). This is `evo_ape`'s real default, used deliberately instead of a looser custom tolerance (see the summary above), but it does mean each run's RMSE reflects a sparser sample than the raw pose counts might suggest.

"GT%" is `slam_coverage_after_fsm_pct` – the SLAM-estimated path length (scaled by the Umeyama factor) divided by the ground-truth path length flown after FSM start. A value near 100% means the backend tracked essentially the whole mission at the correct scale; a value far below 100% would indicate it lost tracking partway through despite a low RMSE.

Table 1: ATE RMSE (mission-only, primary; full-bag, for reference) and tracking coverage, per run and aggregated (mean $\pm$ std across the 3 runs).

Algorithm	Run	Mission ATE (m)	Full-bag ATE (m)	GT% (coverage)
ORB-SLAM2	julien_1	0.1263	0.1661	112.00%
ORB-SLAM2	julien_2	0.1815	0.1896	110.77%
ORB-SLAM2	julien_3	0.1231	0.1231	106.20%
ORB-SLAM2	mean $\pm$ std	0.1436 $\pm$ 0.0328	0.1596 $\pm$ 0.0337	109.66% $\pm$ 3.06%
OV2SLAM (accurate)	julien_1	0.0555	0.1087	101.17%
OV2SLAM (accurate)	julien_2	0.0617	0.1389	100.24%
OV2SLAM (accurate)	julien_3	0.0224	0.0393	101.57%
OV2SLAM (accurate)	mean $\pm$ std	0.0465 $\pm$ 0.0211	0.0956 $\pm$ 0.0511	100.99% $\pm$ 0.68%

On mission ATE, OV2SLAM is substantially tighter than ORB-SLAM2 (0.047m vs. 0.144m mean – roughly $3\times$) on this platform, though both show real run-to-run spread (std 0.021m and 0.033m respectively) given only 3 runs and the small per-run association counts noted above. OV2SLAM's coverage is also tighter around 100% (std 0.68 pts vs. 3.06 pts) – ORB-SLAM2's coverage exceeding 100% by more (106–112%) suggests its scaled trajectory slightly overshoots the true path length, more so than OV2SLAM's.

Why full-bag ATE reads higher. Diagnosed concretely on ORB-SLAM2 julien_1's raw (unassociated) pose stream: the first 20 of 886 raw SLAM poses average 0.337m error against the nearest GT sample; the remaining 866 average 0.104m. Checking the raw ground truth in that early window: `x` sits at exactly 0.000 throughout while `y` swings – i.e. this is the gate's lateral left/right

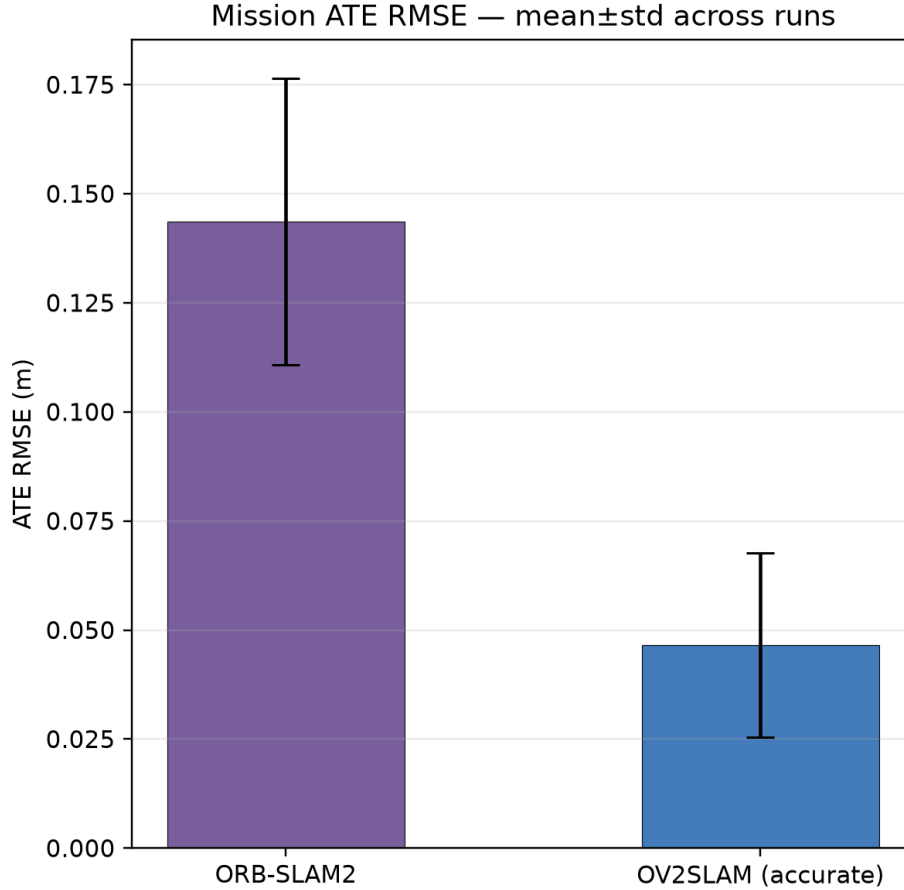


Figure 1: Mission ATE RMSE, mean $\pm$ std across the 3 runs per backend.

warm-up legs (a purely 2D wiggle near the start position), not the mission’s smooth diagonal approach, and it shows up as a short near-vertical segment at the start of the full trajectory plots in Figure 6. SLAM’s pose estimate is naturally noisier in this window (only just initialized, scale/pose not yet settled), so this short segment contributes disproportionately to a whole-bag RMSE – which is exactly why the mission-windowed number above is the one to trust.

3 Front-End Latency

OV2SLAM’s front-end is roughly $3\times$ faster per frame than ORB-SLAM2’s on this platform (8.2 ms vs. 25.7 ms mean).

4 CPU Utilization

Total CPU usage follows the same pattern as latency: OV2SLAM uses roughly less than half the CPU of ORB-SLAM2 on average (33% vs. 77% of one core).

Table 2: Front-end tracking latency per frame, per run and aggregated.

Algorithm	Run	Mean (ms)	Std (ms)
ORB-SLAM2	julien_1	26.902	5.052
ORB-SLAM2	julien_2	24.191	3.396
ORB-SLAM2	julien_3	25.857	4.764
ORB-SLAM2	mean $\pm$ std across runs	25.650	1.367
OV2SLAM (accurate)	julien_1	8.298	3.289
OV2SLAM (accurate)	julien_2	8.807	3.120
OV2SLAM (accurate)	julien_3	7.401	2.907
OV2SLAM (accurate)	mean $\pm$ std across runs	8.169	0.712

5 Per-Thread CPU Breakdown

Data gap – Figure 5 does not show a real per-role breakdown. Unlike ORB-SLAM2, OV2SLAM’s config launched its node via `ros2 run` at the time these 3 runs were recorded. `ros2 run` is a Python wrapper that forks a child process for the real binary instead of exec-replacing itself, so the CPU sampler (which reads `/proc/1/task/`) only ever saw the Python wrapper’s generic OS threads (`ros2`, `bash`, `ros2-ust`) – not OV2SLAM’s actual internal threads. **This has since been fixed**: OV2SLAM’s C++ source now names its threads at the OS level (`ov2_est`, `ov2_map`, `ov2_lc`, `ov2_main`, `ov2_feed`, `ov2_vizfr`, `ov2_vizkf`, `ov2_merge` – matching the offline EuRoC study’s naming convention), and every OV2SLAM stack config now invokes the compiled binary directly instead of through `ros2 run`. Verified live on the Pi post-fix (see `docs/fixlog`) but **not yet re-flown** – per instruction, this report was not delayed for a re-run. A future run with any OV2SLAM config will produce a real per-role breakdown comparable to Figure 4.

6 Trajectories

7 Notes and Caveats

- $n=3$ runs per backend – enough to see a consistent pattern, not enough for strong statistical claims.
- Only the *accurate* OV2SLAM profile is represented here. The *fast* profile has not yet completed a successful HIL run (its warm-up parallax fell short of OV2SLAM’s initialization threshold in the one attempt made) – see `docs/fixlog` for the diagnosis.
- ATE is computed against the *online* `/slam/pose` stream (what the flight controller would consume live, had it been wired to SLAM – see the boxed note in Section 1), not an offline, retroactively-optimized trajectory.
- Neither backend drives the flight in these 6 runs – see the boxed note in Section 1.
- The OV2SLAM per-thread plot gap (Section 5) is the only known missing data point in this report; everything else reflects the 6 runs as recorded.

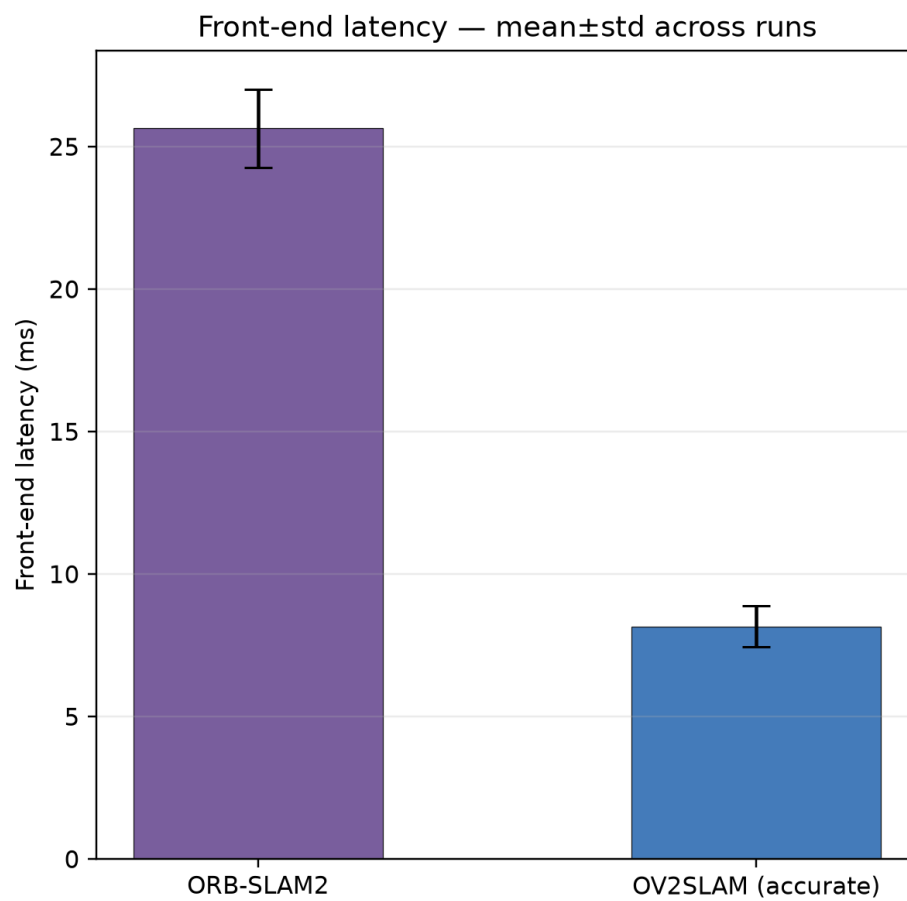


Figure 2: Front-end tracking latency, mean $\pm$ std across the 3 runs per backend.

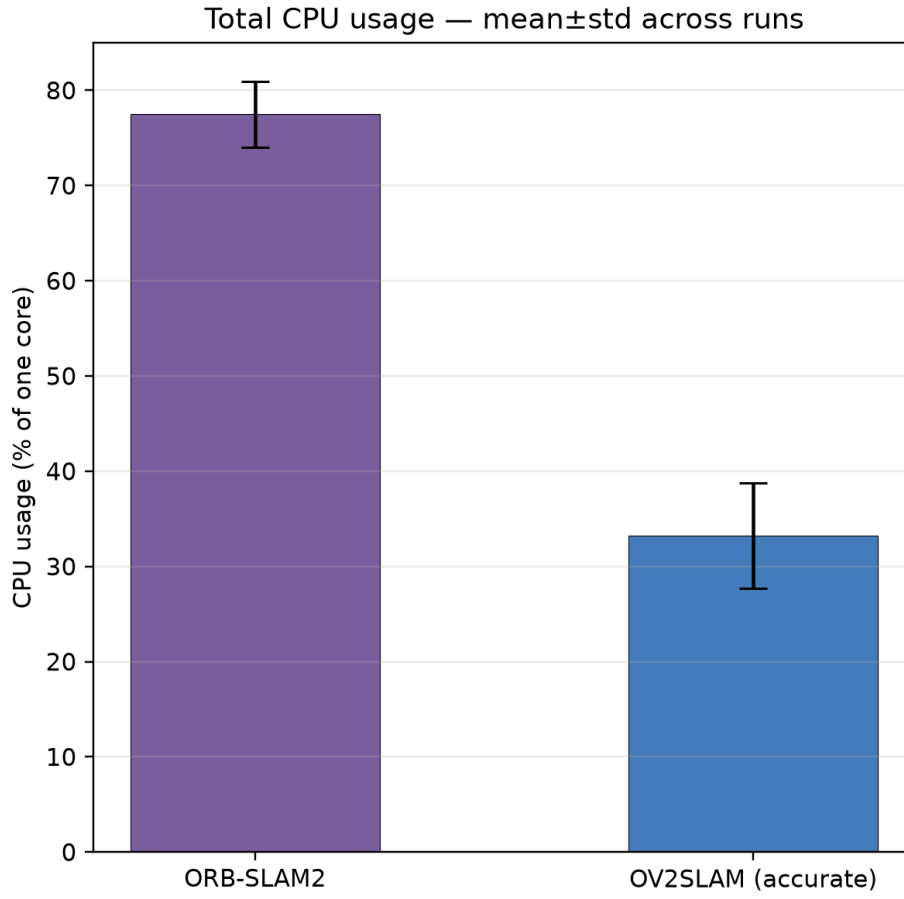


Figure 3: Total process CPU utilization (% of one core), mean $\pm$ std across the 3 runs per backend.

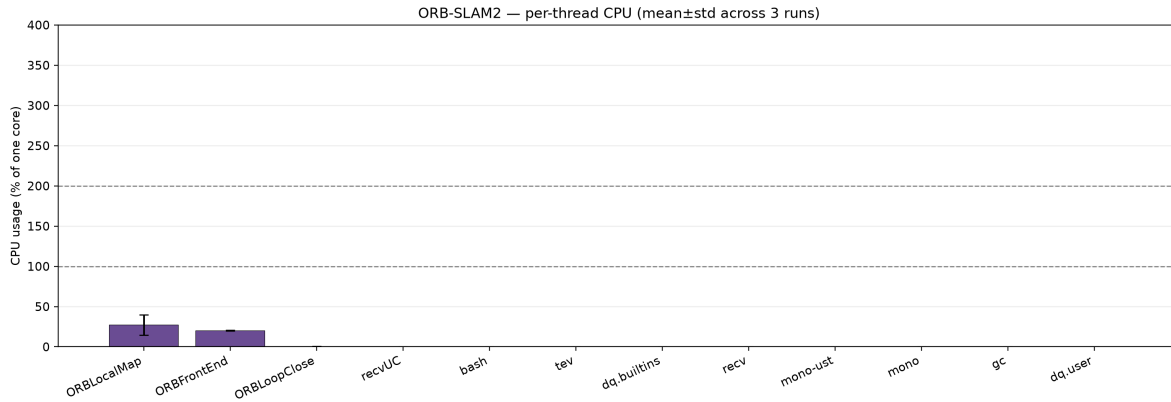


Figure 4: ORB-SLAM2 per-thread CPU usage, mean $\pm$ std across the 3 runs. Named internal roles (ORBLocalMap, ORBFrontEnd, ORBLoopClose) dominate; remaining entries are generic OS/runtime threads.

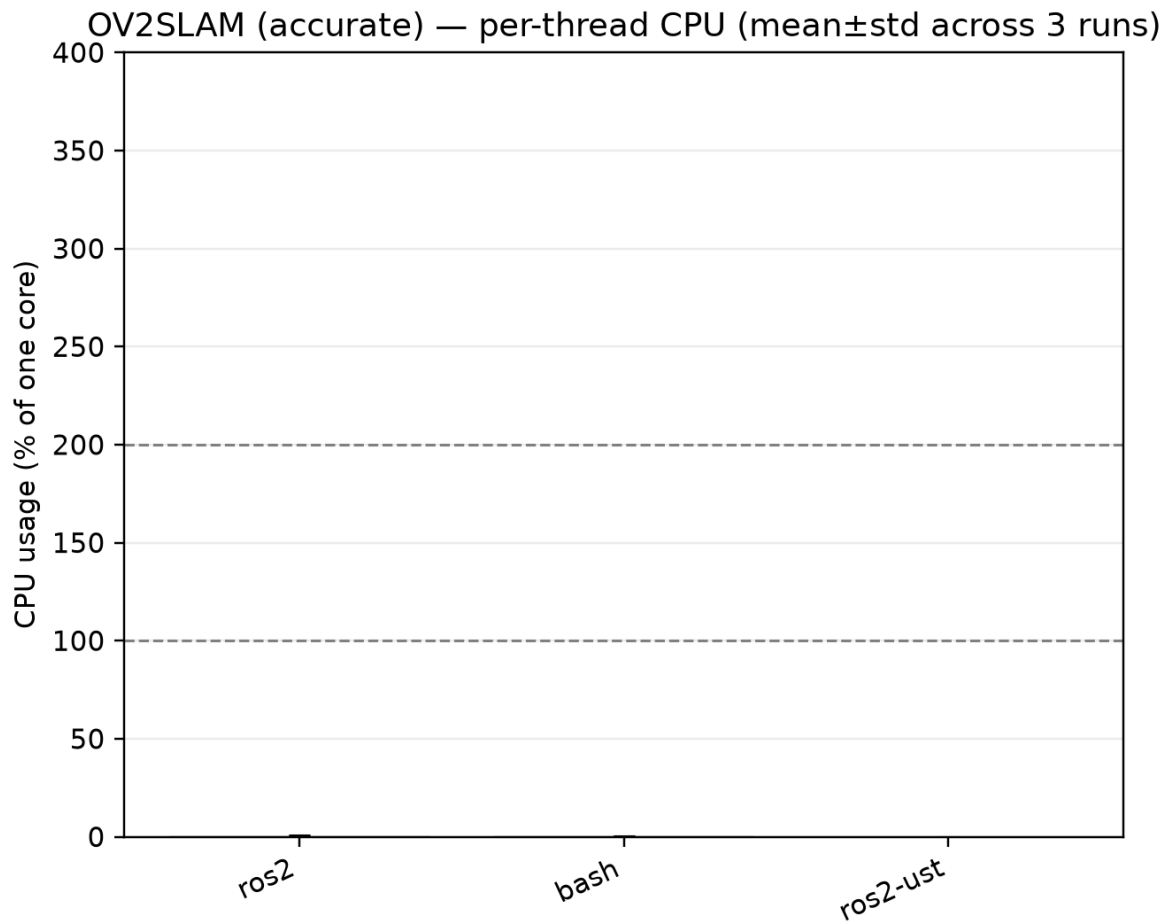


Figure 5: OV2SLAM per-thread CPU usage as captured in these 3 runs.

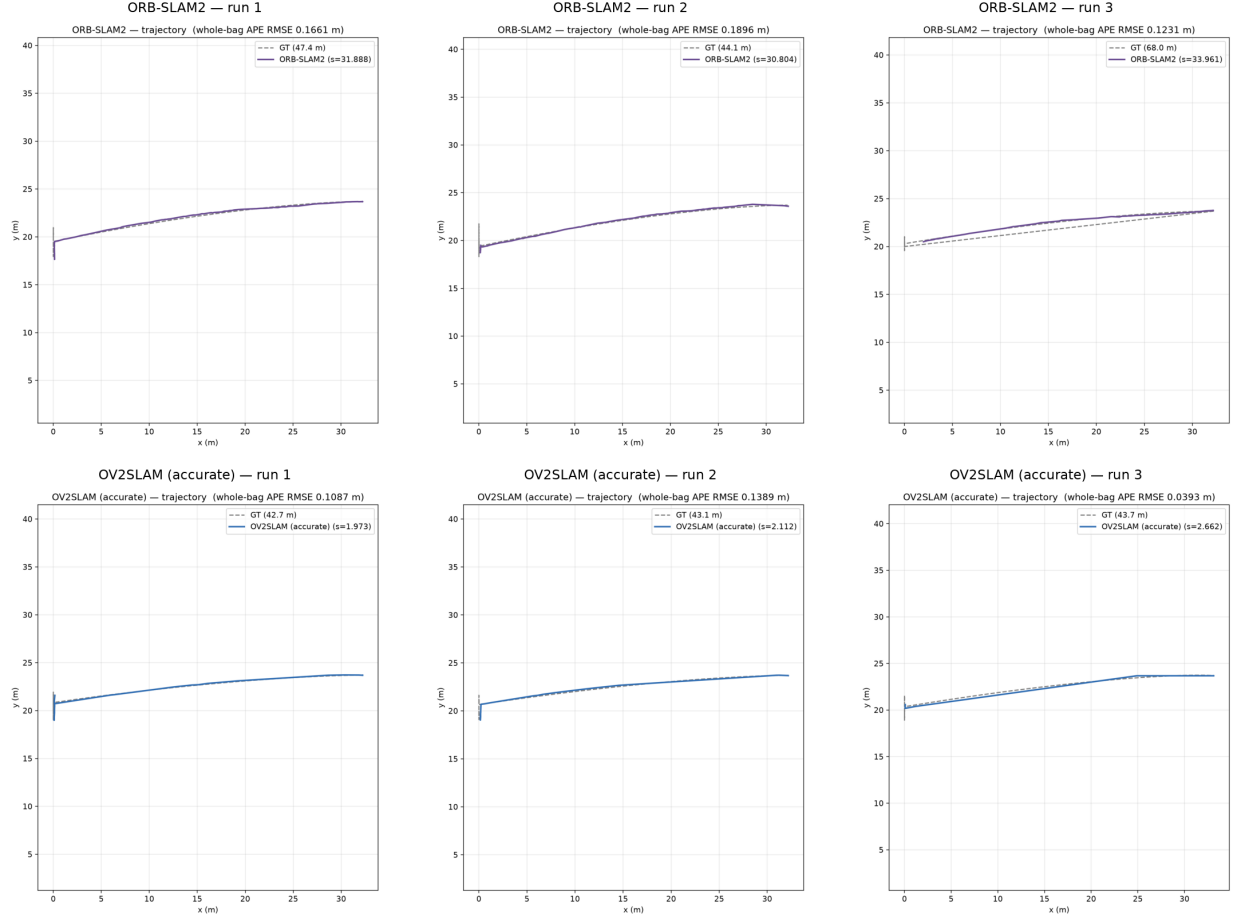


Figure 6: Top-down (x–y) ground-truth vs. SLAM trajectory overlay for all 6 runs (whole bag, including the gate’s pre-mission warm-up). Top row: ORB-SLAM2. Bottom row: OV2SLAM (accurate). Dashed grey: ground truth. Solid: SLAM estimate (Umeyama-aligned, scale s shown in each legend). The short near-vertical segment visible near the start of each trace is the gate’s lateral warm-up wiggle discussed in Section 2 – not a plotting artifact.